

Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

09.03.01 Информатика и вычислительная техника

(направление подготовки/специальность)

Программное обеспечение мобильных систем

(профиль/специализация)

Очная

(форма обучения)

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

(вид практики)

Тип практики Технологическая (проектно-технологическая) практика

на предприятии ООО «Бюро 1440»

(наименование профильной организации/структурного подразделения СибГУТИ)

ТЕМА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

-

Выполнил:

студент института информатики и вычислительной техники Овчаров Ярослав Сергеевич
группа ИА-332

_____/_____
« ____ » _____ 202__ г. (подпись) (ФИО)

Проверил¹

Руководитель практики от профильной
организации

_____/ Андреев А. В. /
(подпись) (ФИО)
« ____ » _____ 202__ г.

Проверил:

Руководитель практики от СибГУТИ

_____/ Брагин К. И. /
(подпись) (ФИО)

« ____ » _____ 202__ г.

отметка ² _____ « ____ » _____ 202__ г.

Новосибирск 2025

¹ В случае прохождения практики в профильной организации

² Заполняется во время промежуточной аттестации

План-график проведения
Производственной практики
вид практики
Мелентьев Владимир Ильич
Фамилия Имя Отчество студента

института ИВТ, курса 3, гр. ИА-332

Направление: 09.03.01 Информатика и вычислительная техника

Код – Наименование направления (специальности)

Направленность (профиль)/ специализация: Программное обеспечение мобильных систем

Место прохождения практики: г. Новосибирск, ул. Бориса Богаткова, д. 51, ауд. 469

Объем практики: 360/10 часов/ЗЕ

Тип практики: Технологическая (проектно-технологическая) практика

Срок практики: с 29.01.2024 по 27.05.2024 (раз в неделю) и с 17.06.2024 по 13.07.2024 (ежедневно);

Содержание практики³:

Тема индивидуального задания практики :-

Наименование видов деятельности	Дата (начало – окончание)
Архитектура Adalm Pluto SDR. Построение радио приемника.	
Установка ПО, Тестирование AdalmPluto с Python. Подготовка TX буфера и передача сэмплов на SDR	
Одновременный прием и передача TX и RX с Adalm Pluto.	
Передача аудиосигнала и его прием с использованием SDR.	

³ В случае прохождения практики в профильной организации

В соответствии с рабочей программой практики
Руководитель практики от профильной
организации*
« 29 » января 2024 г.

_____/ Андреев А. В. /
(подпись) (ФИО)

Руководитель практики от СибГУТИ
« 29 » января 2024 г.

_____/ Брагин К. И. /
(подпись) (ФИО)

Отзыв о работе студента

Добавлено примечание ([КБ1]): Этот лист не трогать!

(ФИО студента)

Уровень освоения компетенций

Компетенции	Уровень сформированности компетенций
ПК-1 Способен разрабатывать требования и проектировать программное обеспечение	
ПК-3 Способен осуществлять эксплуатацию и развитие транспортных сетей и сетей передачи данных, включая спутниковые системы	

Уровень компетенций: высокий, средний, низкий, не сформирована

Руководитель практики от СибГУТИ:

Старший преподаватель
Кафедры ТС и ВС

должность руководителя практики подпись

Брагин К.И.

ФИО руководителя практики

« 13 » июля 2024 г.

Практика 1.

АРХИТЕКТУРА ADALM PLUTO SDR.GNU RADIO.ПОСТРОЕНИЕ РАДИО-ПРИЁМНИКА

Цель: Ознакомиться с архитектурой Adalm Pluto SDR и реализовать программу через GNU Radio, которая будет принимать радио.

Краткие теоретические сведения

Software-Defined Radio (SDR)- радиосистема, в которой традиционно аппаратные компоненты (фильтры, модуляторы, демодуляторы, детекторы, кодеры и т.п.) реализованы на программном уровне, с использованием универсального оборудования и цифровой обработки сигналов.

GNURadio - это инструмент с открытым исходным кодом для разработки программного обеспечения в сфере программно-определяемого радио. Инструмент позволяет при помощи “строительных блоков” создать конфигурацию радиоустройства, не написав ни одной строчки кода, и запустить программу непосредственно с использованием модуля SDR

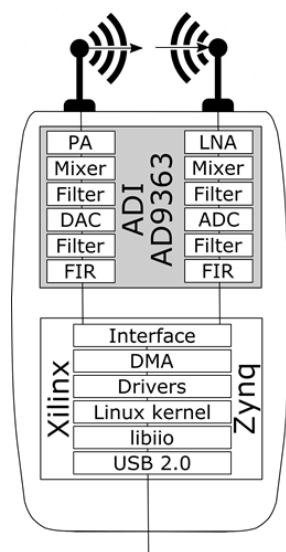


Рис.1 Устройство Adalm Pluto

PA(Power Amplifier) Функция: Усиление сигнала до уровня, требуемого для излучения.

LNA(LowNoise Amplifier) Функция: Усиление слабого приёмного сигнала с минимальным добавлением шума перед его дальнейшей обработкой.

ADC/DAC ADC(Analog-to-Digital Conversion): оцифровка аналогового сигнала в цифровой поток для последующей цифровой обработки. DAC(Digital-to-Analog Conversion): преобразование семплов в аналоговый сигнал перед микшированием.

FIR (Finite Impulse Response) цифровых Функция: детальная фильтрация, коррекция формы спектра, компенсация искажений.

Mixer

TX: перенос низкочастотного сигнала (baseband) на несущую частоту (subcarrier) для отправки в эфир.

RX: перенос высокочастотного сигнала на низкочастотный для дальнейшей обработки.

Filter

TX: фильтрация выходного сигнала перед передачей, подавление лишних гармоник.

RX: фильтрация принимаемого сигнала, подавление внеполосных помех перед оцифровкой.

libiio Описание: библиотека, которая облегчает работу с устройствами ввода/вывода в Linux, особенно с радиочипами серии AD936x. Она даёт API для обмена данными и управления устройствами.

Linux Kernel Описание: ядро Linux, управляющее процессором, памятью и устройствами ввода-вывода. **DMA(Direct Memory Access)** Описание: механизм, который автоматически переносит большие объёмы данных между устройством и памятью, разгружая процессор.

Drivers Описание: инструкции для ядра, объясняющие, как правильно пользоваться конкретным оборудованием.

Xilinx Zynq Описание: семейство микросхем от компании Xilinx. На одном кристалле объединены ARM-процессор (обычно на 2 ядра), который запускает Linux, управляет периферией, и ПЛИС(FPGA) для реализации аппаратных блоков (фильтры, ускорители обработки сигналов) и для более скоростных вычислений в реальном времени.

USB2.0 Описание: порт для связи хоста и Adalm Pluto

Выполнение:

1. Построить блок-схему радиосистемы
2. Настроить параметры
3. Проверить прием радио на SDR

Блок схема состоит из следующих элементов:

QT GUI Range -Графический элемент управления (ползунок) для изменения параметров в реальном времени. Позволяет настраивать частоты, коэффициенты усиления и другие параметры без остановки потока

PlutoSDR Source- Источник сигнала для платы SDR ADALM-PLUTO. Принимает сигналы с радиочастотного входа PlutoSDR. Настраивается на определенную частоту дискретизации и центральную частоту

Low Pass Filter- Фильтр нижних частот (ФНЧ). Пропускает низкие частоты и ослабляет высокие. Используется для выделения полезного сигнала и подавления помех

QT GUI Frequency Sink(3 блока)- задает спектральное представление сигнала, которое меняется в реальном времени.

WBFM Receive- Демодулятор широкополосной частотной модуляции (FM)

Преобразует FM-радиосигнал в аудиосигнал

QT GUI Time Sink- задает временное представление сигнала, которое меняется в реальном времени.

Audio sink- Выходной аудиоблок Воспроизводит обработанный аудиосигнал через звуковую карту компьютера Преобразует цифровой сигнал в аналоговый звук

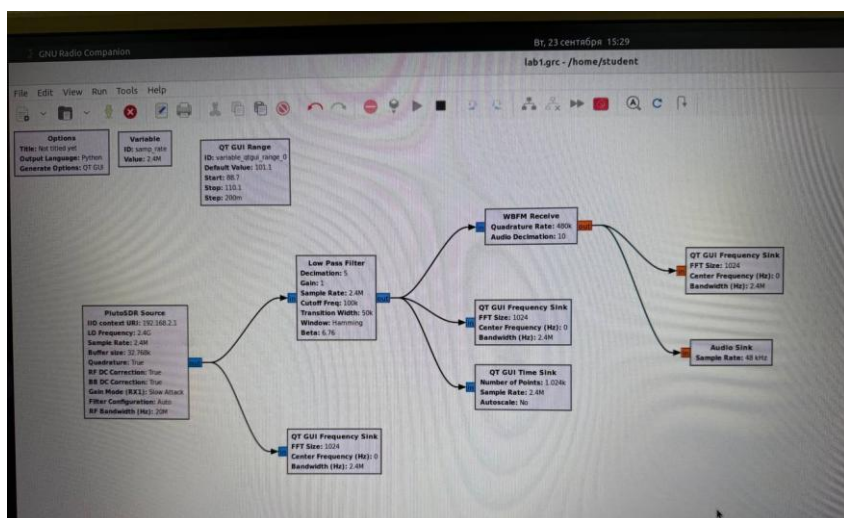


Рис 2.Блок схема радиосистемы

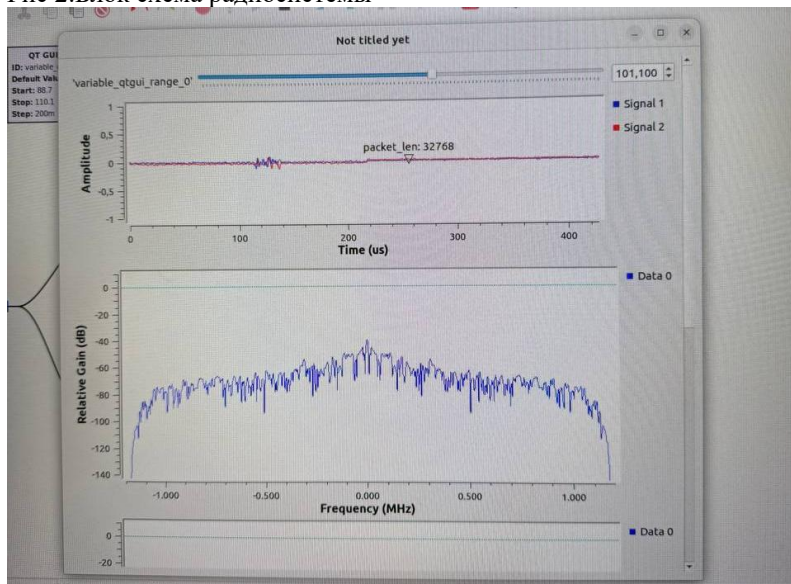


Рис.3 График временной и частотной временной радиостанции

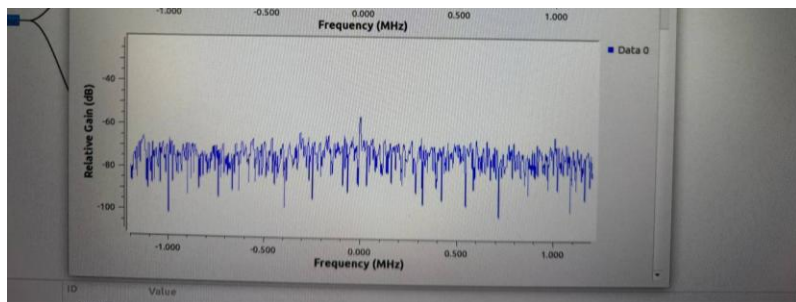


Рис.4 Частотный график

Вывод

Изучил структуру SDR на базовом уровне . Познакомился с GNU Radio и с помощью него построил блок схему, которая принимает FM радио сигнал

Практика 2.

Установка ПО, Тестирование AdalmPluto с Python. Подготовка TX буфера и передача сэмплов на SDR

Цели работы:

1. Практически проверить работоспособность связки Python + AdalmPluto, получив и отобразив первые IQ-сэмплы с приемника.

Краткие теоретические сведения

Основы Python

1. **Python** — интерпретируемый язык программирования высокого уровня с простым и понятным синтаксисом. Широко используется в научных вычислениях, анализе данных и для работы с аппаратурой.
2. **Переменные и типы данных:** Python использует динамическую типизацию. Данные представляются в виде объектов, которые могут быть изменяемыми (mutable, напр., list) и неизменяемыми (immutable, напр., int, tuple).
3. **Списки (list):** Упорядоченные изменяемые коллекции объектов произвольных типов. Для работы с ними используются методы: append(), pop(), sort() и др.
4. **NumPy:** Фундаментальная библиотека для научных вычислений в Python. Предоставляет поддержку многомерных массивов и матриц, а также математические функции для их обработки. Ключевой объект — numpy.ndarray.
5. **Matplotlib:** Библиотека для визуализации данных и создания статических, анимированных и интерактивных графиков.

Программно-определяемое радио (SDR) и AdalmPluto

- **AdalmPluto (PlutoSDR)** — малогабаритный SDR-модуль, способный и передавать, и принимать радиосигналы в диапазоне 70 МГц – 6 ГГц.
- **IQ-данные (In-Phase / Quadrature):** Для обработки сигналов в цифровом виде используется квадратурное представление. Сигнал описывается двумя компонентами: I (синфазной) и Q (квадратурной). В комплексной форме это записывается как $I + j*Q$. Это позволяет однозначно определить амплитуду и фазу сигнала.
- Формат CS16 - 16-битные целочисленные значения для I и Q компонент

1. Библиотеки для работы с PlutoSDR:

1. **libiio** — низкоуровневая кроссплатформенная библиотека для взаимодействия с аппаратными интерфейсами Analog Devices.
2. **pyadi-iio** — Python-привязка к libiio, предоставляющая удобный высокоуровневый API для управления устройствами, включая PlutoSDR.

Ход выполнения работы

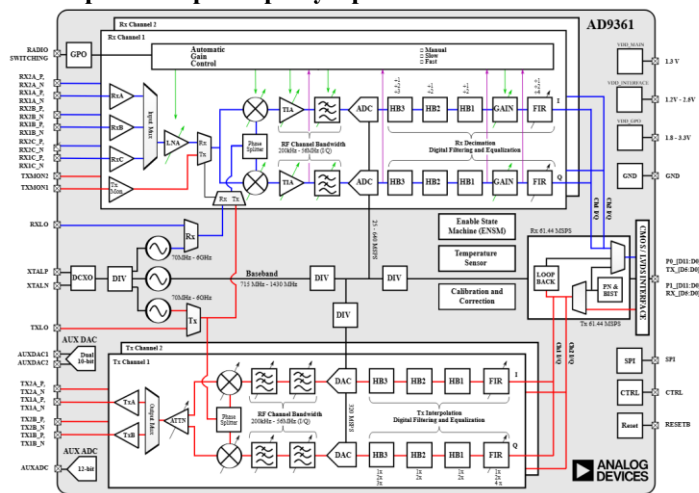
Выполнение заданий на Python

Задание: Изучение Matplotlib

Цель: Построение графиков гармонических сигналов.

Выполнение:

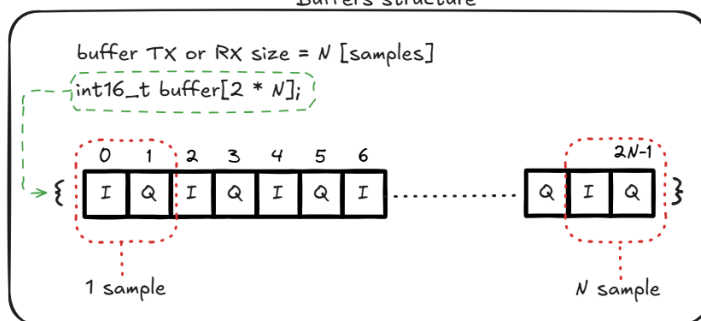
1. Начнем с инициализации устройства SoapySDR
2. Настройка параметров устройств TX/RX



3. Инициализация потоков (stream) для передачи и приема сэмплов

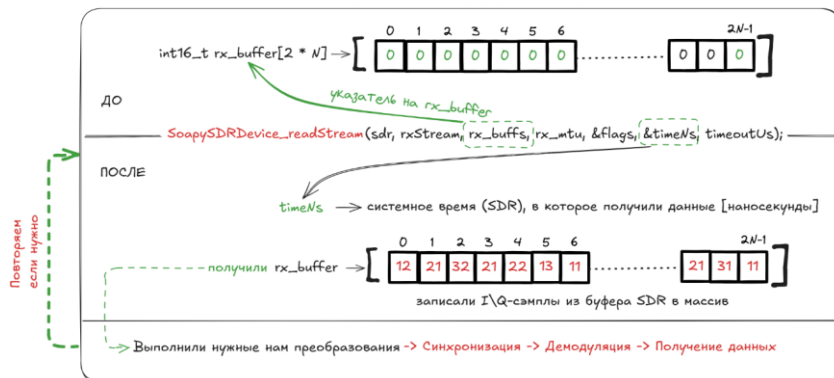
4. Выделение памяти под буферы (для хранения сэмплов TX, RX):

Buffers structure



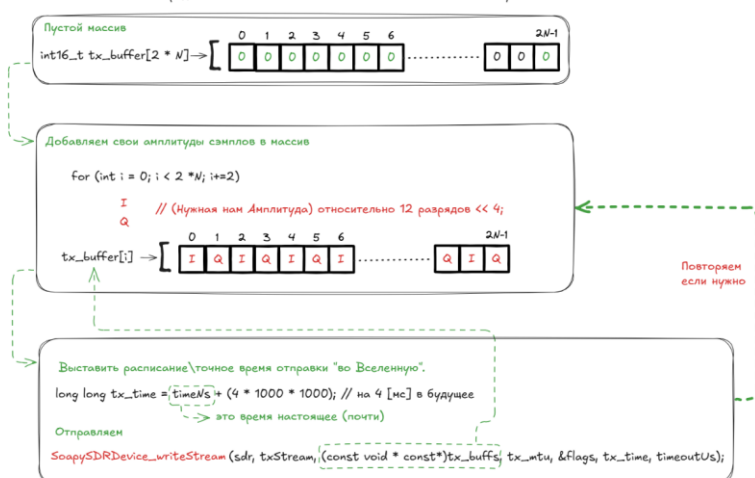
5. Чтение сэмплов с SDR

Чтение сэмплов с "Вселенная -> Антенна -> SDR"



6. Подготовка массива для передачи (TX buffer)

Передача своих сэмплов на "SDR -> Антенна -> Во вселенную"



Результат:

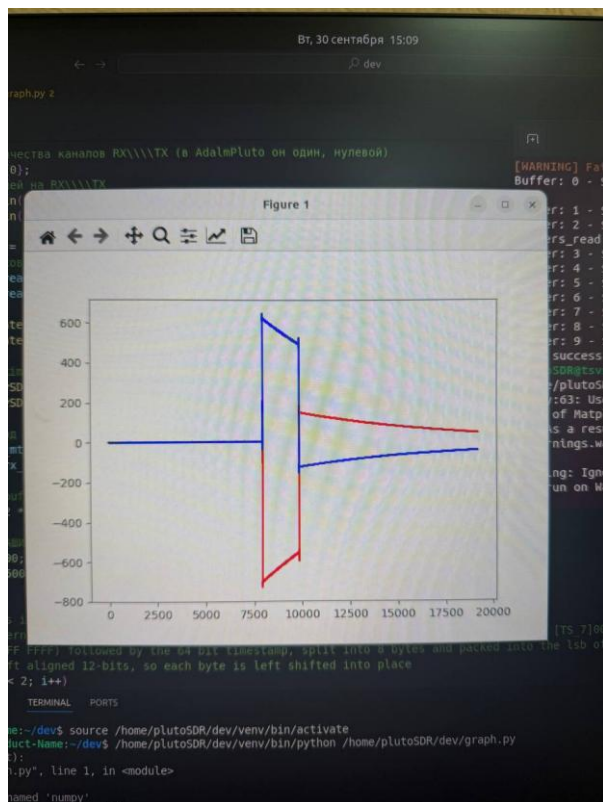


Рис.5

Графическое отображение передачи сэмплов на SDR

Сам код

Вывод: научился передавать сэмплы их TX буферы на SDR и строить их график

Практика 3.

Одновременный прием и передача TX и RX с Adalm Pluto.

Цель работы

1. Реализовать одновременный прием и передачу данных через SDR-устройство Adalm Pluto.
2. Освоить принципы синхронизации RX и TX потоков с использованием временных меток.
3. Изучить организацию циклических буферов для непрерывной обработки данных.
4. Настроить систему сборки проекта с помощью CMake.

Краткие теоретические сведения

Архитектура одновременного приема-передачи

PlutoSDR поддерживает полно-дуплексный режим работы, позволяя одновременно принимать и передавать данные на одной частоте. Ключевые аспекты:

1. **Временные метки (timestamps)** - обеспечивают точную синхронизацию операций
2. **Буферизация данных** - циклические буферы для непрерывного потока
3. **MTU (Maximum Transmission Unit)** - оптимальный размер блока данных

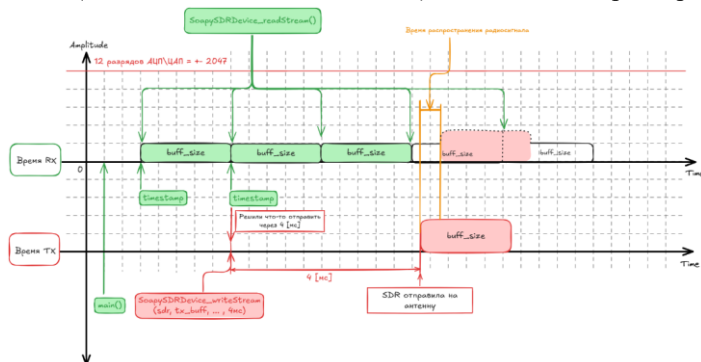
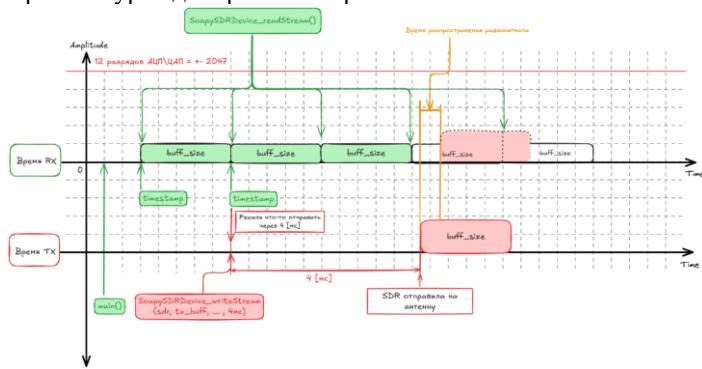


Рис.6
Схема работы буфера

Ход выполнения работы

1. Идентичен предыдущей практике
2. Архитектура одновременной работы



3. Для сборки проекта можно воспользоваться системой сборки – Сmake

Результаты:

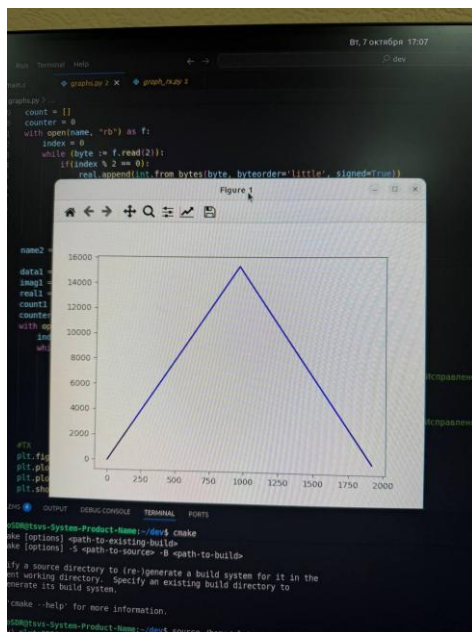


Рис.7

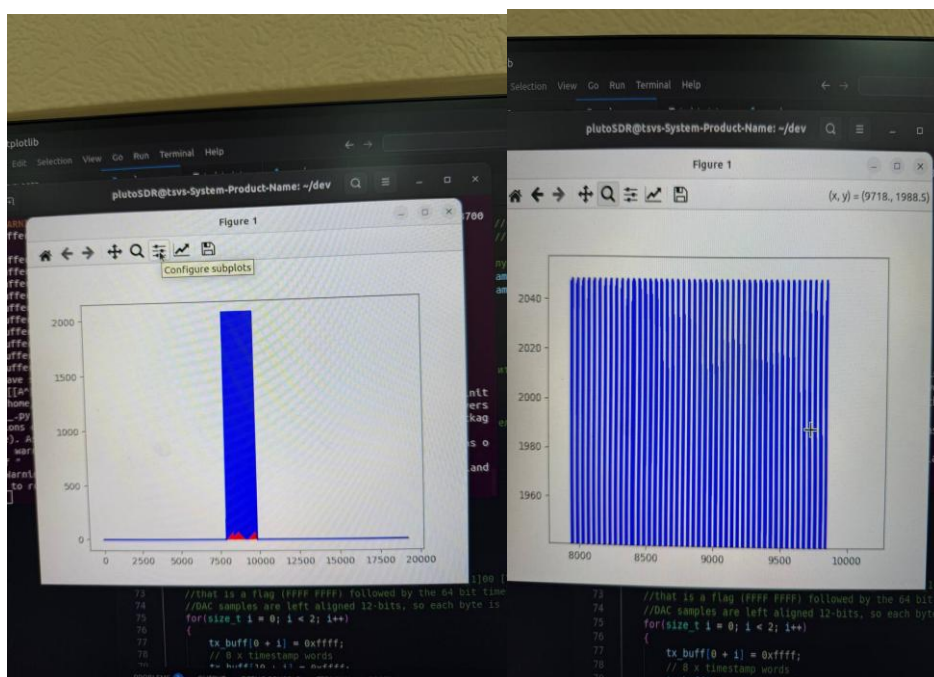


Рис.8
Одновременная передача rx tx

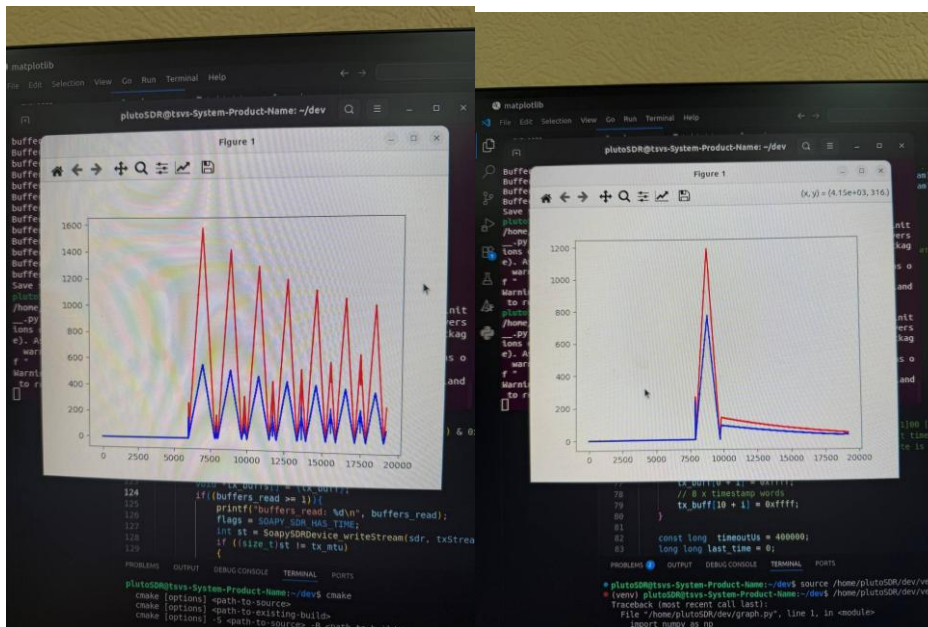


Рис.9
Одновременная передача tx и rx

Вывод: научился Одновременно передавать и принимать TX и RX с Adalm Pluto.

Практика 4.

Передача аудиосигнала и его прием с использованием SDR.

Цель работы

1. Освоить преобразование аудиосигналов между форматами MP3 и PCM.
2. Реализовать передачу аудиоданных через SDR-устройство Adalm Pluto.
3. Организовать прием аудиосигнала и его сохранение в файл.
4. Создать полную цепочку обработки аудио: MP3 → PCM → RF → PCM → MP3.

Краткие теоретические сведения

- **MP3** - сжатый формат с потерями, использует психоакустическое сжатие
- **PCM (Pulse Code Modulation)** - несжатый цифровой формат, представляет аналоговый сигнал в виде последовательности отсчетов.
- Характеристики PCM:
 - Частота дискретизации: 44.1 кГц (стандарт для аудио)
 - Разрядность: 16 бит
 - Диапазон значений: -32768 до +32767

Архитектура системы передачи

MP3 файл → PCM данные → Модуляция → RF передача → RF прием →
→ Демодуляция → PCM данные → MP3 файл.

Ход выполнения работы

1. Подключение библиотек

Первым шагом подключаются необходимые библиотеки для работы со звуком и массивами данных:

```
import numpy as np
import librosa
from pydub import AudioSegment
```

- **numpy** — для работы с числовыми массивами;
- **librosa** — для загрузки MP3-файлов и преобразования их в массив отсчётов;
- **pydub** — для экспорта и кодирования звуков в различные форматы (в том числе MP3).

2. Функция конвертации MP3 → PCM

Загрузка аудио

```
y, sr = librosa.load(mp3_file, sr=44100, mono=True)
```

- `y` — массив отсчётов (float32 в диапазоне [-1.0, 1.0]);
- `sr` — частота дискретизации (обычно 44100 Гц).

Преобразование к 16-битным значениям

```
pcm_data = (y * 32767).astype(np.int16)
```

- Значения масштабируются к диапазону [-32768, 32767];
- Тип данных изменяется на `int16`, как в стандартном PCM.

Запись в файл

```
pcm_data.tofile(pcm_file)
```

Создаётся бинарный файл без заголовков, содержащий только "сырые" отсчёты.

3. Функция конвертации PCM → MP3

Теперь реализуем обратное преобразование — из несжатого PCM обратно в MP3.

Чтение данных:

```
pcm_data = np.fromfile(pcm_file, dtype=np.int16)
```

Считываем весь бинарный файл в виде массива 16-битных чисел

Создание объекта AudioSegment:

```
audio = AudioSegment(
    data=pcm_data.tobytes(),
    sample_width=2,          # 2 байта = 16 бит
    frame_rate=44100,        # частота дискретизации
    channels=1                # моно
)
```

Создаётся объект звука, понятный библиотеке `pydub`.

Экспорт в MP3:

```
audio.export(mp3_file, format="mp3", bitrate="192k")
```

Данные сохраняются в формате MP3 с заданным битрейтом 192 кбит/с.

4. Передача и прием на SDR

1. Чтение из файла.

```
int16_t *read_pcm(const char *filename, size_t *sample_count)
{
    FILE *file = fopen(filename, "rb");
```

```

fseek(file, 0, SEEK_END);
long file_size = ftell(file);
fseek(file, 0, SEEK_SET);
printf("file_size = %ld\n", file_size);
int16_t *samples = (int16_t *)malloc(file_size);

*sample_count = file_size / sizeof(int16_t);

size_t sf = fread(samples, sizeof(int16_t), *sample_count, file);

if (sf == 0){
    printf("file %s empty!", filename);
}

fclose(file);

return samples;
}

```

2. Передать семплы

```

void *tx_buffs[] = {tx_buff};
int st = SoapySDRDevice_writeStream(
    device->sdr, device->txStream,
    (const void *)tx_buffs,
    BLOCK_SAMPLES,
    &flags_tx, tx_time, timeoutUs
);

```

tx_buffs - массив указателей, в нашем случае указатель один на первый элемент (двигая указатель можем передавать по блокам).

3. Принять семплы

```

int sr = SoapySDRDevice_readStream(
    device->sdr, device->rxStream,
    rx_buffs, device->rx_mtu,
    &flags_rx, &timeNs, timeoutUs
);

```

4. Записать в файл

```

int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs, rx_mtu, &flags, &timeNs,
timeoutUs);

fwrite(rx_buffer, sizeof(int16_t), 2 * rx_mtu, file2);

```

В результате получаем изначальный аудиофайл, так же мы дополнительно провели эксперимент передав сигнал одновременно с двух SDR, тем самым получили искаженные аудиофайлы и изучили явление **интерференции**

Вывод: Научились передавать аудиофайл и принимать его при помощи SDR, так же изучили явление интерференции.